MINOR PROJECT 2
ARYAN SINGH

Problem Selection: Credit Card Transaction Anomaly Detection

Relevant Real-World Problem

Credit card fraud causes financial loss for banks, merchants, and customers. A major challenge is that fraud labels are often unavailable, delayed, incomplete, or highly imbalanced. Therefore, instead of training a model only on known fraud examples, an unsupervised machine learning system can learn the pattern of normal transactions and flag transactions that are significantly different.

This project uses **Credit Card Transaction Anomaly Detection** to identify suspicious transactions without depending on fraud labels.

Problem Statement

Financial institutions process thousands of credit card transactions every day. Manually checking every transaction is impractical, while fraudulent transactions may look different from a customer's usual spending behavior.

The problem is to detect unusual transactions by analyzing features such as:

- Transaction amount
- Transaction time
- Merchant category
- Location distance from the customer's usual area
- Number of transactions in a short time period
- Transaction frequency
- Device or payment mode, if available

Since fraud labels may not be available, the system will use unsupervised learning algorithms to identify transactions that do not match normal transaction patterns.

Project Objective

The objective of this project is to build an unsupervised machine learning model that detects potentially suspicious credit card transactions.

The project will:

- Clean and preprocess transaction data.
- Create meaningful behavioral features such as high amount, unusual transaction time, location distance, and frequent transactions.

MINOR PROJECT 2
ARYAN SINGH

- Apply Isolation Forest, DBSCAN, and optionally One-Class SVM to detect anomalies.
- Assign an anomaly score or anomaly label to each transaction.
- Display the most unusual transactions for investigation.
- Explain possible reasons for flagging a transaction, such as unusually high spending, abnormal timing, or excessive transaction frequency.
- Compare the performance and usefulness of the selected algorithms using metrics such as anomaly count, cluster separation, and visual analysis.

This solution can help banks prioritize transactions for manual review and reduce the risk of fraudulent activity.

DATASET SELECTION:

I have used a dataset from kagglehub as it has diverse features and very close to real world data.

The datasets contains transactions made by credit cards in September 2013 by European cardholders. This dataset present transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions.

DATASET LINK: https://www.kaggle.com/datasets/joebeachcapital/credit-card-fraud

right now just give the steos and outcomes

3. Data Preprocessing

The selected dataset contains mostly numerical features (`Time`, `V1`–`V28`, and `Amount`), so preprocessing will mainly focus on data quality checks, duplicate removal, transformation of skewed values, feature scaling, and exploratory analysis. The `Class` column will be excluded from model training because this is an unsupervised anomaly-detection project.

Step 1: Load and Inspect the Dataset

MINOR PROJECT 2
ARYAN SINGH

Load the CSV file and inspect its structure using functions such as `head()`, `info()`, `describe()`, and `shape`.

Expected outcome:
The dataset is expected to contain approximately 284,807 transaction records and 31 columns: `Time`, `V1` to `V28`, `Amount`, and `Class`.

Step 2: Check Data Types

Verify that all columns have suitable numeric data types. `Time`, `V1`–`V28`, and `Amount` should be numerical.

Expected outcome:
No categorical columns are expected, so feature encoding will not be required.

Step 3: Handle Missing Values

Check every column for missing values using `isnull().sum()`.

Expected outcome:
The commonly used Kaggle version of this dataset is expected to have no missing values. If any missing values are found in the downloaded version, numerical values will be filled using the median of the respective column, because the median is less affected by extreme transaction values.

Step 4: Remove Duplicate Records

Check for duplicate transaction rows using `duplicated().sum()` and remove exact duplicates if present.

Expected outcome:
Duplicate records, if found, will be removed to prevent the same transaction from influencing the anomaly model multiple times. The final dataset will contain only unique transactions.

Step 5: Separate the Target Label

Store the `Class` column separately and remove it from the training features.

Expected outcome:
The anomaly-detection models will train only on transaction characteristics, without using fraud labels. The stored `Class` column may be used later only for optional validation of detected anomalies.

Step 6: Transform the `Amount` Feature

MINOR PROJECT 2
ARYAN SINGH

Analyze the distribution of `Amount`, which is usually highly right-skewed because a small number of transactions have very large values. Apply a transformation such as `log1p(Amount)` if needed.

Expected outcome:
The transformed amount feature will have a less skewed distribution, reducing the excessive influence of very high-value transactions while still preserving their unusual nature.

Step 7: Process the `Time` Feature

Convert `Time` from seconds into more interpretable features if useful, such as:

- Hour of the day
- Day indicator
- Cyclical time features using sine and cosine transformation

Expected outcome:
The model will be able to identify transactions occurring at unusual times more effectively than with raw seconds alone.

Step 8: Feature Encoding

Encoding is not required because the dataset contains only numerical and anonymized features.

Expected outcome:
No one-hot encoding or label encoding will be performed. All input features are already in numerical form.

Step 9: Feature Scaling / Normalization

Scale all input features using `StandardScaler` or `RobustScaler`. This is especially important because `Amount` and `Time` have much larger ranges than `V1`–`V28`.

Expected outcome:
All features will be placed on a comparable scale, preventing high-range columns from dominating distance-based algorithms such as DBSCAN and One-Class SVM. Scaling will also improve the reliability of Isolation Forest, DBSCAN, and One-Class SVM.

Step 10: Exploratory Data Analysis (EDA)

Perform EDA to understand transaction patterns before model building. The analysis will include:

- Dataset dimensions and column summary
- Descriptive statistics: mean, median, minimum, maximum, and standard deviation
- Distribution plots for `Amount` and transformed amount
- Distribution of transaction time
- Box plots to identify extreme values

- Correlation heatmap for numerical features
- Scatter plots or PCA-based 2D visualization to observe transaction grouping
- Optional comparison of normal and fraud records using `Class`, only for understanding the dataset and not for training

Expected outcome:
EDA will reveal whether transaction amounts are skewed, whether outliers exist, whether certain features are correlated, and whether transactions show visible clusters or isolated points. These observations will guide the choice of anomaly-detection algorithm and hyperparameters.

MODEL DEVELOPMENT

4. Model Development

For this project, Isolation Forest will be used as the main unsupervised machine learning model for detecting suspicious credit card transactions.

Isolation Forest is highly suitable for this problem because it is designed specifically for anomaly detection. Unlike clustering algorithms, it does not need to create fixed groups of transactions. Instead, it identifies unusual transactions by measuring how easily they can be isolated from the majority of normal transactions. Suspicious transactions are usually isolated in fewer random splits than normal transactions.

Model Training Process

The cleaned and scaled transaction features obtained after preprocessing will be used as input for the model. The `Class` column will be excluded from the training dataset because the project follows an unsupervised learning approach.

The model will be trained using the following features:

- Processed transaction time features
- Transformed and scaled transaction amount
- Anonymized transaction features `V1` to `V28`

The Isolation Forest model will be configured using parameters such as:

- `n_estimators`: Number of isolation trees to build, for example 100 or 200
- `contamination`: Estimated proportion of anomalous transactions, for example 0.01 or 0.02
- `random_state`: A fixed value to ensure reproducible results

After training, the model will predict whether each transaction is normal or anomalous.

- `1` will represent a normal transaction.
- `-1` will represent a suspicious or anomalous transaction.

The model will also generate an anomaly score for every transaction. Transactions with lower scores will be considered more unusual and will receive higher priority for review.

Expected Model Output

The trained model will produce a new column named `Anomaly_Label` and another column named `Anomaly_Score`.

Transaction ID	Transaction Amount	Anomaly Score	Anomaly Label	Status
1024	High amount	Low score	-1	Suspicious
1025	Normal amount	High score	1	Normal
1026	Unusual transaction pattern	Very low score	-1	High-priority suspicious

Identification of Suspicious Transactions

Transactions predicted as `-1` will be filtered and stored as suspicious transactions. The suspicious transactions will be ranked based on their anomaly scores.

The top unusual transactions will be analyzed using available features such as:

- Transaction amount significantly higher or lower than typical transactions
- Transaction occurring at an unusual time
- Transaction having an uncommon combination of anonymized feature values
- Transaction being far from the normal transaction pattern learned by the model

Expected Outcome

The Isolation Forest model will identify transactions that differ significantly from common transaction behavior. The final system will provide a ranked list of suspicious transactions along

with anomaly scores, allowing financial institutions to prioritize transactions for manual investigation.

Conclusion

This project focuses on detecting suspicious credit card transactions using an unsupervised machine learning approach. Since fraud labels are often unavailable or delayed in real-world financial systems, the model does not use the `Class` column during training. Instead, it learns the overall pattern of normal transactions and identifies records that differ significantly from that pattern.

After cleaning the dataset, removing duplicate records, transforming relevant features, and scaling the transaction data, the Isolation Forest algorithm will be trained to detect anomalies. Isolation Forest is suitable for this project because it can efficiently process a large number of transaction records and assign an anomaly score to each transaction.

The final model will classify transactions as normal or suspicious. Transactions with low anomaly scores will be considered more unusual and can be prioritized for manual review. The detected anomalies may represent transactions with unusually high amounts, uncommon transaction timing, or rare combinations of transaction features.

Overall, this project demonstrates how unsupervised machine learning can support fraud-risk detection without requiring labelled fraud data during training. The system can help financial institutions reduce manual effort, identify potentially risky transactions early, and improve the efficiency of transaction monitoring. In the future, the project can be improved by adding customer location, merchant category, device information, transaction frequency, and real-time transaction data to provide more accurate and explainable anomaly detection.

1. Semantic Segmentation vs Instance Segmentation vs Panoptic Segmentation

Semantic Segmentation classifies every pixel in an image into a category. All objects belonging to the same category are given the same label. For example, if an image contains three cars, all car pixels will be labelled as “car,” but the model will not distinguish between Car 1, Car 2, and Car 3.

MINOR PROJECT 2
ARYAN SINGH

Instance Segmentation also classifies every pixel, but it separates different objects of the same category. For example, if an image contains three cars, each car receives a separate identity, such as Car 1, Car 2, and Car 3. It is useful when identifying individual objects is important.

Panoptic Segmentation combines semantic and instance segmentation. It assigns a class label to every pixel and also separates individual objects where needed. For example, road and sky are labelled as background regions, while each person and car is identified as a separate object instance.

Type	Pixel Classification	Separates Same-Type Objects?	Example
Semantic Segmentation	Yes	No	All cars are labelled “car”
Instance Segmentation	Yes	Yes	Each car gets a different label
Panoptic Segmentation	Yes	Yes, for object instances	Road is one region; each car and person is separate

2. What is Fuzzy Logic? How Is It Different from Boolean Logic?

Fuzzy Logic is a method of reasoning in which a value can have a degree of truth between 0 and 1. Instead of saying something is completely true or completely false, fuzzy logic allows partial truth.

For example, in a temperature-control system, a temperature of 30°C may be considered “warm” with a membership value of 0.7 and “hot” with a membership value of 0.3. This makes fuzzy logic useful for real-world situations where categories are not always exact.

Boolean Logic works only with two fixed values:

- `0` or False
- `1` or True

For example, in Boolean Logic, a room is either “hot” or “not hot.” There is no middle value.

MINOR PROJECT 2
ARYAN SINGH

Feature	Boolean Logic	Fuzzy Logic
Truth values	Only 0 or 1	Any value between 0 and 1
Decision type	Exact and fixed	Gradual and flexible
Example	Temperature is hot / not hot	Temperature is slightly hot, warm, or very hot
Best use	Digital circuits, exact conditions	Washing machines, AC systems, traffic control, decision systems

3. Customer Dataset Without Labels

Suppose a dataset contains customer **Age**, **Annual Income**, and **Spending Score**, but no target labels.

1. Is this supervised or unsupervised learning?

This is an **unsupervised learning** problem because no output label or target variable is provided. The aim is to discover hidden patterns or groups of similar customers based on their age, income, and spending behavior.

2. Which clustering algorithm would you choose first and why?

The first algorithm to use would be **K-Means Clustering**.

K-Means is suitable because the dataset contains numerical features and the objective is to group similar customers into segments. It is simple, fast, easy to implement, and easy to explain in a report. It can create useful customer groups such as:

- Young customers with high spending scores
- High-income customers with low spending scores
- Low-income customers with moderate spending scores
- Older customers with stable spending patterns

Before applying K-Means, the features should be scaled using StandardScaler because Annual Income, Age, and Spending Score may have different ranges.

The Elbow Method and Silhouette Score can be used to choose the most suitable number of clusters, represented by K.

3. What assumptions does K-Means make about the data?

K-Means makes the following assumptions:

- The data can be divided into a predefined number of clusters, K.
- Each cluster is roughly spherical or circular in shape.
- Customers within the same cluster are closer to each other than to customers in other clusters.
- Clusters are separated mainly based on distance, usually Euclidean distance.
- Features should be on a similar scale; otherwise, a large-range feature such as Annual Income may dominate the clustering result.
- Clusters are expected to have somewhat similar size and density.
- Extreme outliers can affect cluster centroids and reduce clustering quality.

Therefore, K-Means is a strong first choice for this customer segmentation dataset, but the data should be scaled and checked for outliers before training the model.